



TIT TECHNOCRATS



Only Group of MP & CG with NBA in 3 Engg. Colleges

**Central India's
Largest Technical
Educational Group**



UNBEATABLE PLACEMENTS

Technocrats Group Campus, BHEL, Bhopal - 462021 MP, India

Phone : +91-755-2751679 | Mobile : +91-9826374295, +91-9893141968

e-mail : placements@titbhupal.net | website : <http://www.technocratgroup.edu.in>



TIT TECHNOCRATS



**UNBEATABLE
PLACEMENTS**

Campus Specific Training

By: Prof. Gautam Singh

Technocrats Group of Institutions

Technocrats Group Campus, BHEL, Bhopal - 462021 MP, India

Phone : +91-755-2751679

e-mail: info@titbhopal.net | website: www.technocratsgroup.edu.in

1. Introduction to Java Collections Framework

- What is the Java Collection Framework?
- Need for Collection Framework
- Advantages over Arrays
- Collection Framework Hierarchy
- Key Interfaces: **Collection**, **List**, **Set**, **Queue**, **Map**

1. Introduction to Java Collections Framework

What is the Java Collection Framework?

The **Java Collection Framework (JCF)** is a set of **interfaces and classes** that provide ready-to-use data structures for storing and processing collections of objects efficiently. It is part of the `java.util` package and includes Lists, Sets, Queues, Maps, and utility classes.

Need for Collection Framework

Before Java introduced Collections, developers used **arrays** to store multiple objects. However, arrays have limitations:

1. **Fixed Size** - Once an array is created, its size cannot be changed.
2. **No Built-in Methods** - Arrays lack built-in methods for common operations like sorting, searching, and inserting.
3. **Performance Issues** - Operations like inserting elements in the middle are inefficient.

The **Collection Framework** solves these problems by providing **dynamic data structures** and a variety of **utility methods**.

- ◆ **List** → Ordered, allows duplicates (e.g., `ArrayList`, `LinkedList`).
- ◆ **Set** → Unordered, does not allow duplicates (e.g., `HashSet`, `TreeSet`).
- ◆ **Queue** → Follows FIFO (e.g., `PriorityQueue`, `ArrayDeque`).

1 Collection Interface (Root Interface)

```
import java.util.*;
```

```
public class CollectionExample {  
    public static void main(String[] args) {  
        Collection<String> names = new ArrayList<>();  
        names.add("John");  
        names.add("Emma");  
        System.out.println(names); // Output: [John, Emma]  
    }  
}
```

② List Interface (Ordered, Duplicates Allowed)

```
import java.util.*;
```

```
public class ListExample {  
    public static void main(String[] args) {  
        List<Integer> list = new ArrayList<>();  
        list.add(10);  
        list.add(20);  
        list.add(10);  
        System.out.println(list); // Output: [10, 20, 10]  
    }  
}
```

③ Set Interface (Unordered, No Duplicates)

```
import java.util.*;
```

```
public class SetExample {  
    public static void main(String[] args) {  
        Set<String> set = new HashSet<>();  
        set.add("Apple");  
        set.add("Banana");  
        set.add("Apple"); // Duplicate, will be ignored  
        System.out.println(set); // Output: [Apple, Banana]  
    }  
}
```


4 Queue Interface (FIFO)

```
import java.util.*;
```

```
public class QueueExample {  
    public static void main(String[] args) {  
        Queue<String> queue = new  
LinkedList<>();  
        queue.add("First");  
        queue.add("Second");  
        System.out.println(queue.poll());  
    }  
}
```


5 Map Interface (Key-Value Pairs)

```
import java.util.*;
```

```
public class MapExample {  
    public static void main(String[] args) {  
        Map<Integer, String> map = new HashMap<>();  
        map.put(1, "One");  
        map.put(2, "Two");  
        System.out.println(map.get(1)); // Output: One  
    }  
}
```

ArrayList

- Creating an `ArrayList`
- Adding, Removing, Updating Elements
- Iterating over an `ArrayList` (`forEach`, `Iterator`, `ListIterator`)
- Sorting using `Collections.sort()`
- Performance Comparison with `LinkedList`

What is ArrayList?

ArrayList is a **dynamic array implementation** of the **List** interface in Java. Unlike arrays, an **ArrayList** can grow and shrink in size dynamically.

- ✓ **Ordered Collection** – Maintains insertion order
- ✓ **Allows Duplicates** – Can contain duplicate elements
- ✓ **Fast Access** – Provides **O(1)** time complexity for **get(index)**
- ✓ **Resizable** – No fixed size, unlike arrays

Creating an ArrayList

```
import java.util.ArrayList;
```

```
public class ArrayListExample {  
    public static void main(String[] args) {  
        // Creating an ArrayList of Strings  
        ArrayList<String> list = new ArrayList<>();
```

```
        // Adding elements  
        list.add("Apple");  
        list.add("Banana");  
        list.add("Cherry");
```

```
        System.out.println(list); // Output: [Apple, Banana, Cherry]  
    }  
}
```

Adding, Removing, and Updating Elements

✓ Adding Elements

`list.add("Mango");` // Adds at the end
`list.add(1, "Orange");` // Inserts at index 1

✗ Removing Elements

`list.remove("Banana");` // Removes "Banana"
`list.remove(2);` // Removes element at index 2

↻ Updating Elements

`list.set(1, "Grapes");` // Replaces index 1 with "Grapes"

Iterating over an ArrayList:-

① Using forEach (Java 8+)

```
list.forEach(System.out::println);  
import java.util.ArrayList;
```

```
public class ForEachExample {  
    public static void main(String[] args) {  
        ArrayList<String> list = new ArrayList<>();  
        list.add("Apple");  
        list.add("Banana");  
        list.add("Cherry");  
  
        // Using forEach with a method reference  
        list.forEach(System.out::println);  
  
        // Using forEach with a lambda expression  
        list.forEach(item -> System.out.println("Fruit: " + item));  
    }  
}
```

② Using for-loop

```
for (int i = 0; i < list.size(); i++) {  
    System.out.println(list.get(i));  
}
```

```
import java.util.ArrayList;
```

```
public class ForLoopExample {  
    public static void main(String[] args) {  
        ArrayList<String> list = new ArrayList<>();  
        list.add("Apple");  
        list.add("Banana");  
        list.add("Cherry");  
  
        // Using for-loop to iterate over the list  
        for (int i = 0; i < list.size(); i++) {  
            System.out.println(list.get(i));  
        }  
    }  
}
```


③ Using Iterator

```
import java.util.Iterator;
```

```
Iterator<String> iterator = list.iterator();  
while (iterator.hasNext()) {  
    System.out.println(iterator.next());  
}
```

```
import java.util.ArrayList;  
import java.util.Iterator;
```

```
public class IteratorExample {  
    public static void main(String[] args) {  
        ArrayList<String> list = new ArrayList<>();  
        list.add("Apple");  
        list.add("Banana");  
        list.add("Cherry");  
  
        // Creating an Iterator  
        Iterator<String> iterator = list.iterator();  
  
        // Iterating over the list  
        while (iterator.hasNext()) {  
            System.out.println(iterator.next());  
        }  
    }  
}
```

4) Using ListIterator (Bidirectional Traversal)

```
import java.util.ListIterator;
```

```
ListIterator<String> listIterator = list.listIterator();  
while (listIterator.hasNext()) {  
    System.out.println(listIterator.next());  
}
```

```
import java.util.ArrayList;
import java.util.ListIterator;

public class ListIteratorExample {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");

        // Creating a ListIterator
        ListIterator<String> listIterator = list.listIterator();

        System.out.println("Forward Traversal:");
        while (listIterator.hasNext()) { // Moving Forward
            System.out.println(listIterator.next());
        }

        System.out.println("\nBackward Traversal:");
        while (listIterator.hasPrevious()) { // Moving Backward
            System.out.println(listIterator.previous());
        }
    }
}
```

Sorting using Collections.sort()

To sort an `ArrayList`, use `Collections.sort()`:

```
import java.util.*;

public class SortArrayList {

    public static void main(String[] args) {

        ArrayList<Integer> numbers = new ArrayList<>();

        numbers.add(30);

        numbers.add(10);

        numbers.add(20);

        Collections.sort(numbers); // Sort in ascending order

        System.out.println(numbers); // Output: [10, 20, 30]

        Collections.sort(numbers, Collections.reverseOrder()); // Sort in descending order

        System.out.println(numbers); // Output: [30, 20, 10]

    }

}
```

Example: Fast Insertions in LinkedList

```
import java.util.LinkedList;
public class LinkedListExample {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();

        // Add elements
        list.add("Apple");
        list.addFirst("Banana"); // Adds at the beginning
        list.addLast("Cherry"); // Adds at the end

        System.out.println(list); // Output: [Banana, Apple, Cherry]

        // Remove elements
        list.removeFirst(); // Removes "Banana"
        list.removeLast(); // Removes "Cherry"

        System.out.println(list); // Output: [Apple]
    }
}
```



TIT TECHNOCRATS



UNBEATABLE PLACEMENTS



TIT TECHNOCRATS



UNBEATABLE PLACEMENTS



TIT TECHNOCRATS



UNBEATABLE PLACEMENTS



TIT TECHNOCRATS



UNBEATABLE PLACEMENTS



TIT TECHNOCRATS



UNBEATABLE PLACEMENTS



TIT TECHNOCRATS



UNBEATABLE PLACEMENTS



TIT TECHNOCRATS



UNBEATABLE PLACEMENTS



TIT TECHNOCRATS



UNBEATABLE PLACEMENTS